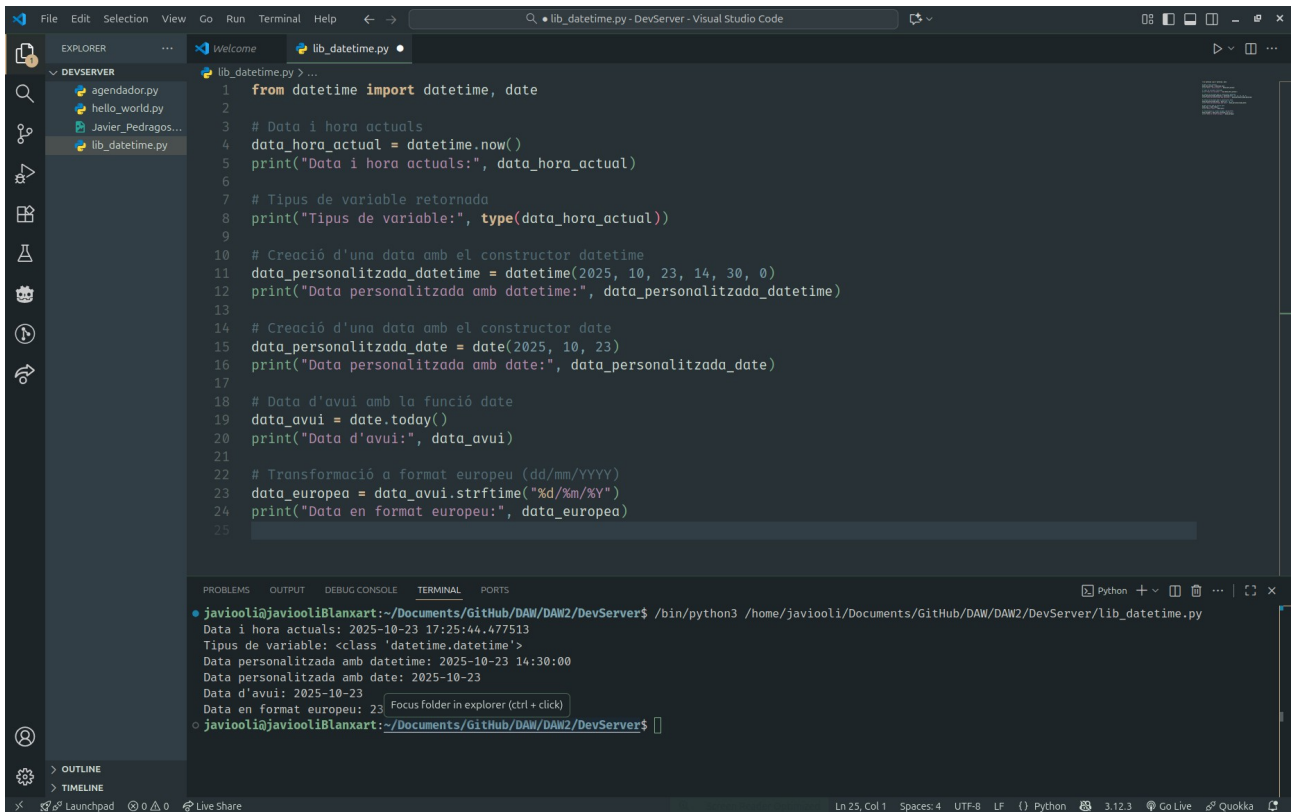
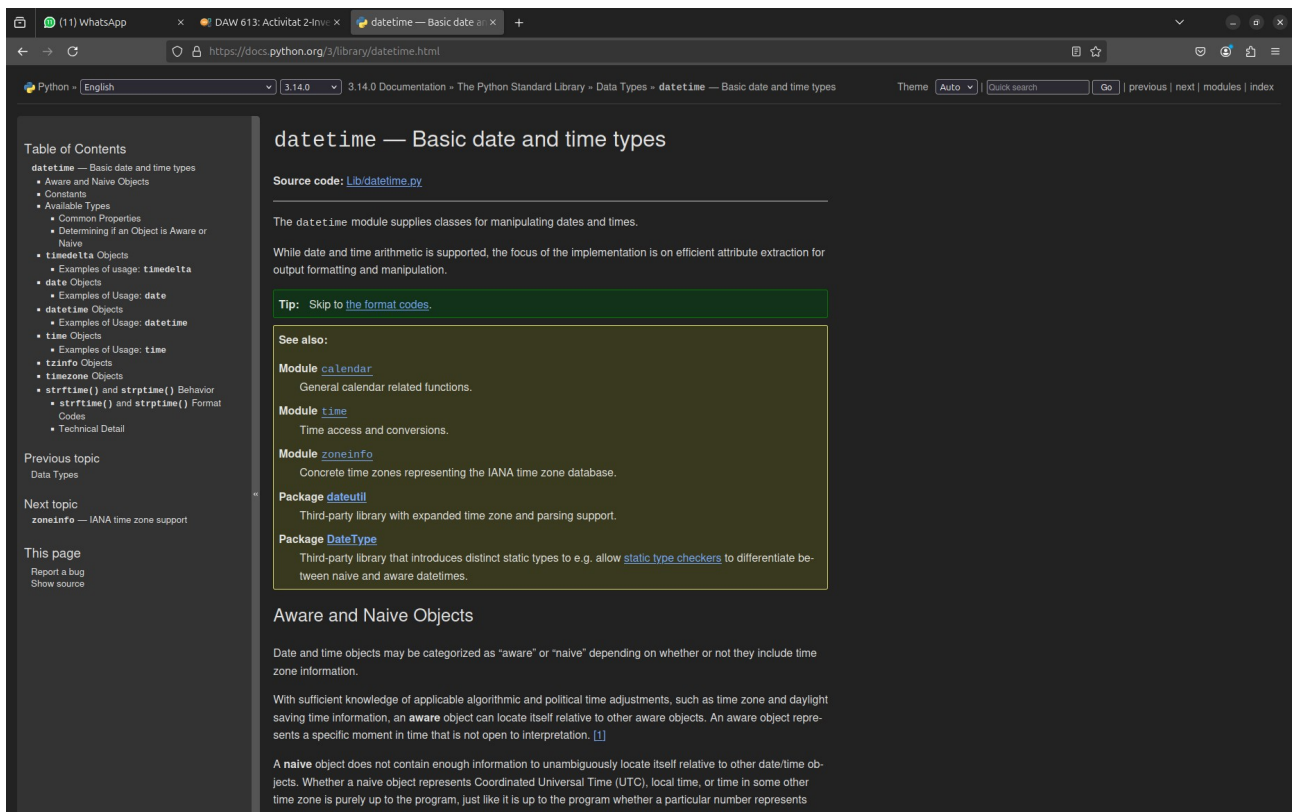


Investigació de la llibreria datetime



```
1 from datetime import datetime, date
2
3 # Data i hora actuals
4 data_hora_actual = datetime.now()
5 print("Data i hora actuals:", data_hora_actual)
6
7 # Tipus de variable retornada
8 print("Tipus de variable:", type(data_hora_actual))
9
10 # Creació d'una data amb el constructor datetime
11 data_personalitzada_datetime = datetime(2025, 10, 23, 14, 30, 0)
12 print("Data personalitzada amb datetime:", data_personalitzada_datetime)
13
14 # Creació d'una data amb el constructor date
15 data_personalitzada_date = date(2025, 10, 23)
16 print("Data personalitzada amb date:", data_personalitzada_date)
17
18 # Data d'avui amb la funció date
19 data_avui = date.today()
20 print("Data d'avui:", data_avui)
21
22 # Transformació a format europeu (dd/mm/YYYY)
23 data_europea = data_avui.strftime("%d/%m/%Y")
24 print("Data en format europeu:", data_europea)
25
```

```
javioli@javioliBlanxart:~/Documents/GitHub/DAW/DAW2/DevServer$ /bin/python3 /home/javioli/Documents/GitHub/DAW/DAW2/DevServer/lib_datetime.py
Data i hora actuals: 2025-10-23 17:25:44.477513
Tipus de variable: <class 'datetime.datetime'>
Data personalitzada amb datetime: 2025-10-23 14:30:00
Data personalitzada amb date: 2025-10-23
Data d'avui: 2025-10-23
Data en format europeu: 23/10/2025
javioli@javioliBlanxart:~/Documents/GitHub/DAW/DAW2/DevServer$
```



datetime — Basic date and time types

Source code: [Lib/datetime.py](#)

The datetime module supplies classes for manipulating dates and times.

While date and time arithmetic is supported, the focus of the implementation is on efficient attribute extraction for output formatting and manipulation.

Tip: Skip to [the format codes](#).

See also:

- Module [calendar](#)**
General calendar related functions.
- Module [time](#)**
Time access and conversions.
- Module [zoneinfo](#)**
Concrete time zones representing the IANA time zone database.
- Package [dateutil](#)**
Third-party library with expanded time zone and parsing support.
- Package [DateType](#)**
Third-party library that introduces distinct static types to e.g. allow [static type checkers](#) to differentiate between naive and aware datetimes.

Aware and Naive Objects

Date and time objects may be categorized as “aware” or “naive” depending on whether or not they include time zone information.

With sufficient knowledge of applicable algorithmic and political time adjustments, such as time zone and daylight saving time information, an **aware** object can locate itself relative to other aware objects. An aware object represents a specific moment in time that is not open to interpretation. [1]

A **naive** object does not contain enough information to unambiguously locate itself relative to other date/time objects. Whether a naive object represents Coordinated Universal Time (UTC), local time, or time in some other time zone is purely up to the program, just like it is up to the program whether a particular number represents metres, miles, or more. Naive objects are easy to understand and to work with, at the cost of ignoring some as

Petit programa

```
File Edit Selection View Go Run Terminal Help
agendador.py - DevServer - Visual Studio Code

EXPLORER
agendador.py
hello_world.py
Javier_Pedragos...
lib_datetime.py

DEVSERVER
agendador.py > ...
lib_datetime.py

1 import datetime
2
3 #
4 # Mètodes Utilitat
5 #
6
7 Funció recursiva per rebre entrada per consola i validar-la.
8 No retorna res.
9
10 def ask(msg, validator) -> None:
11     #
12     # Mètodes interns
13     #
14     Funció que valida que l'input sigui un String vàlid.
15     Retorna 0 o str
16
17 def validate_string(entrada):
18     if entrada.strip() == "0": # Si és 0, retornem 0
19         return 0
20
21     if not isinstance(entrada, str): # Comprovem que sigui String
22         return "Invalid"
23
24     if len(entrada.strip()) <= 0: # Comprovem que no sigui buit
25         return "Invalid"
26
27     try: # Comprovem que no sigui numèric.
28         float(entrada)
29     except ValueError:
30         return entrada
31
32     return "Invalid"
33
34
35 Funció que valida que l'input sigui una data vàlida.
36 Retorna 0 o datetime.date
37
38 def validate_date(entrada):
39     YEAR_FORMAT = [False, "200", "20", "2", ""]
40     if entrada.strip() == "0": # Si és 0, retornem 0
41         return 0
42
43     try: # Intentem castejar, en cas d'excepció l'entrada no és una data vàlida.
44         entrada = entrada.split('/')
45         entrada[2] = f'{YEAR_FORMAT[len(entrada[2])]}{entrada[2]}'
46         return datetime.date(int(float(entrada[2])), int(float(entrada[1])), int(float(entrada[0])))
47     except (IndexError, TypeError, ValueError):
48         return "Invalid"
49
50 #
51 # Lògica de la funció
52 #
53
54 VALIDATIONS = {"string": validate_string, "date": validate_date} # Mètode corresponent al tipus d'entrada a validar
55 entrada = VALIDATIONS[validator](input(f'{msg}: ')) # Ex.: validator = string, cridem validate_string(entrada)
56
57 if entrada in ["Invalid"]: # Crida recursiva en cas que l'entrada no sigui vàlida
58     print("Entrada no vàlida.")
59     return ask(msg, validator)
60
61 return entrada
62
63 '''Mètode recursiu que recull activitats i dates fins que s'introdueixi 0.'''
64 def agendar(nom):
65     data = 0
66     activitat = ask(f'Quina activitat vols realitzar, {nom}?', "string")
67
68     if activitat:
69         data = ask(f'Quin dia vols {activitat.lower()}?', "date")
70
71     if data:
72         print("== Activitat registrada ==\n")
73         return f'{activitat} - {data.strftime("%d/%m/%Y")}\n{agendar(nom)}'
74
75     return ""
76
77 #
78 # MAIN
79 #
80 def main():
81     nom = ask("Introdueix el teu nom", "string")
82     agenda = f'\n{agendar(nom)}'
83     print(agenda if agenda != "" else "No s'ha registrat cap activitat")
84     print(f'Adéu, {nom}!')
85
86 if __name__ == "__main__":
87     main()
```